SOLID Principles

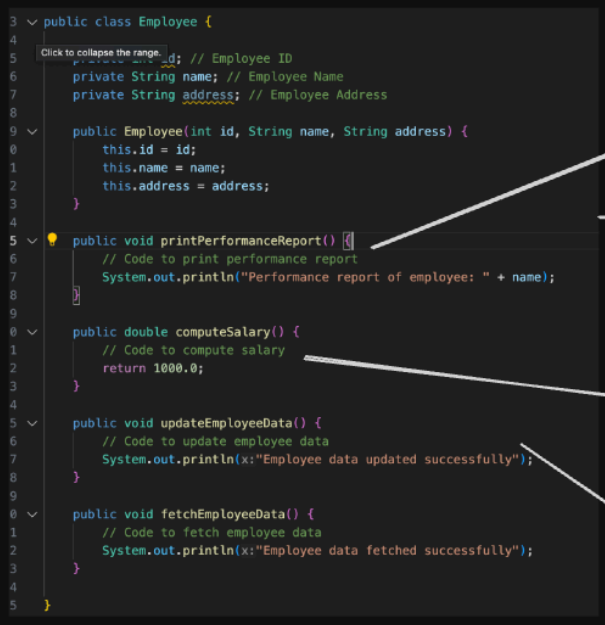
S - Single responsibility principle (SRP)

O - Open Closed Principle (OCP)

L - Liskov substitution Principle (LSP)

I - Interface segregation Principle (ISP)

D - Dependency inversion Principle (DIP)



```java
public class Employee {

    private int id; // Employee ID
    private String name; // Employee Name
    private String address; // Employee Address

    public Employee(int id, String name, String address) {
        this.id = id;
        this.name = name;
        this.address = address;
    }

    public void printPerformanceReport() {
        // Code to print performance report
        System.out.println("Performance report of employee: " + name);
    }

    public double computeSalary() {
        // Code to compute salary
        return 1000.0;
    }

    public void updateEmployeeData() {
        // Code to update employee data
        System.out.println(x:"Employee data updated successfully");
    }

    public void fetchEmployeeData() {
        // Code to fetch employee data
        System.out.println(x:"Employee data fetched successfully");
    }

}
```

If the report format changes, we might need to update this function.

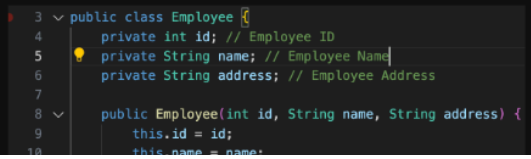
This class is trying to do too many things.
k

Say, if the taxation changes, we might need to update this function.

If data storage requirements change, we might need to update this function

Because the above class is doing too many things, there are too many reasons to update the code present in the class. This violates Single responsibility principle.

Single responsibility principle states that, there should one and only one reason to change a piece of code.



```java
public class Employee {
    private int id; // Employee ID
    private String name; // Employee Name
    private String address; // Employee Address

    public Employee(int id, String name, String address) {
        this.id = id;
```

```java
        this.address = address;
    }

    public int getEmployeeId() {
        return id;
    }

    public String getEmployeeName() {
        return name;
    }

    public String getEmployeeAddress() {
        return address;
    }

    public void setEmployeeAddress(String address) {
        this.address = address;
    }

    public void setEmployeeName(String name) {
        this.name = name;
    }
}
```

This class is now only responsible for Basic employee data creation and fetching

```java
public class EmployeeSalaryCalculator {
    public double computeSalary(Employee e) {
        // Code to compute salary
        return 1000.0;
    }
}
```

This class is only responsible for computing salary

```java
public class EmployeePerformanceReportGenerator {
    public void printPerformanceReport(Employee e) {
        // Code to print performance report
        System.out.println("Performance report of employee: " + e.getEmployeeName
    }
}
```

This class is only responsible for printing performance report

We have segregated the classes in a way such that their is one core responsibility every class has, hence there is only one reason to change logic in a class.

OCP - Open closed principle

A class should be open for extensions but closed for modifications.

```java
public class NotificationSender {

    public void sendNotifications(List<String> notificationTypes, String message) {
        for (String notificationType : notificationTypes) {
            switch (notificationType) {
                case "EMAIL":
                    EmailNotification notification = new EmailNotification();
                    notification.sendEmailNotification(message);
                    break;
                case "SMS":
                    SMSNotification smsNotification = new SMSNotification();
                    smsNotification.sendSMSNotification(message);
                    break;
                case "PUSH":
                    PushNotification pushNotification = new PushNotification();
                    pushNotification.sendPushNotification(message);
                    break;
```

This class will be modified every time we introduce or remove any type of notification.

This is a clear violation of OCP

```
        }
    }
}
```

How to improve ? We should try to think that if futher alteration in the requirements can unnecesary impact code of a class, it is violating OCP

```java
public interface Notification {

    void sendMessage(String message);
}

```

Interface to represent any notification

Individual type of notification implementing the interface

```java
public class EmailNotification implements Notification {

    @Override
    public void sendMessage(String message) {
        System.out.println("Email notification: " + message);
    }
}

```

```java
public class PushNotification implements Notification {

    @Override
    public void sendMessage(String message) {
        System.out.println("Push notification: " + message);
    }
}

```

```java
public class NotificationSender {

    public void sendNotifications(List<Notification> notifications, String message) {
        for (Notification notification : notifications) {
            notification.sendMessage(message);
        }
    }

}

```

Instead of being tightly coupled to type of notifications, we now depend on Notification interface.

Even if we introduce new type of notifications or remove any older one, the sender class is not impacted.

It is a good idea to depend on abstractions rather than concrete classes.